BÀI THỰC HÀNH SỐ 4: CASE STUDY CÀI ĐẶT KIẾN TRÚC MICROSERVICES TRÊN MÔI TRƯỜNG .NET CORE (BÀI TOÁN ĐẠI LÝ BẢO HIỂM)

Phần 2: CÀI ĐẶT MÔI TRƯỜNG

KIẾN TRÚC THAM KHẢO THEO YÊU CẦU CỦA BÀI THỰC HÀNH SỐ 3: Xem phụ lục ở cuối file.

Dựa vào kiến trúc này, sinh viên thực hiện các task tiếp theo như sau:

Task #1: QUY TẮC TỔ CHỨC SOURCE CODE

A) QUY TẮC

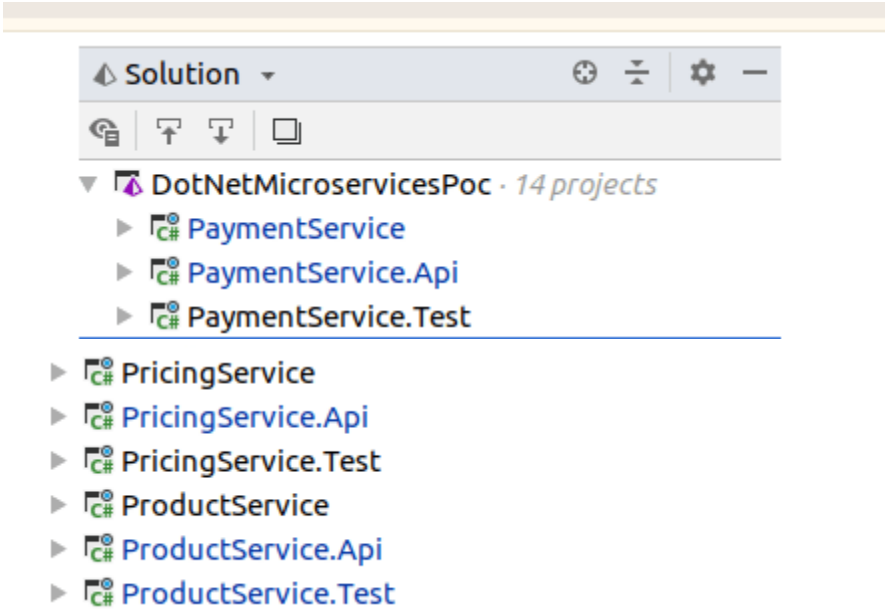
- Mỗi microservice, chúng ta tạo ra ba dự án: một dự án định nghĩa API, một dự án triển khai và một dự án kiểm thử.
- Cụ thể:
    - Dự án định nghĩa API (API definition project) chứa các lớp và giao diện mô tả khả năng của dịch vụ được hiển thị ra thế giới bên ngoài dưới dạng các lệnh mà nó có thể xử lý, các truy vấn mà nó có thể trả lời, các sự kiện mà nó phát ra và các lớp đối tượng truyền dữ liệu mà nó cung cấp. Chúng ta có thể coi đây là các định nghĩa cổng trong kiến trúc cổng và bộ điều hợp.
    - Dự án triển khai (Implementation project) chứa các trình xử lý lệnh, trình xử lý truy vấn và trình xử lý thông báo, cùng nhau cung cấp chức năng của dịch vụ. Hầu hết logic nghiệp vụ nằm trong phần mô hình miền. Các bộ điều hợp để giao tiếp với thế giới bên ngoài được triển khai dưới dạng bộ điều khiển (để xử lý các yêu cầu HTTP đến), trình lắng nghe (cho các sự kiện được gửi qua hàng đợi) và máy khách REST (để xử lý các yêu cầu HTTP đi).
    - Dự án kiểm thử (Test project) chứa cả kiểm thử đơn vị và kiểm thử tích hợp.
- Sử dụng GitHub để quản lý source.

***B)* Nhiệm vụ cụ thể**

1) Tạo SOLUTION trên .NET Core hoặc Rider IDE hoặc các IDE khác.

- Mỗi microservice tạo 3 project theo quy tắc nêu trên (chỉ tạo tên, chưa viết code).

Có thể tham khảo ví dụ sau:



2) Tạo tài khoản Github trên web và cài GitHub Desktop vào máy tính.
3) Add Thầy vào dự án để có thể get source code về kiểm tra và chấm bài, email: caotxuan@gmail.com

Task #2: CÀI ĐẶT MỘT SỐ CODE BAN ĐẦU

1) **Cài MediatR**

- Dùng **Package Manager Console** để thêm MediatR vào dự án.
- Gợi ý:
    - Gõ Install-Package MediatR
    - Đăng ký nó trong DI container chỉ bằng cách thêm mã: services.AddMediatR() trong lớp Startup ở phương thức: ConfigureServices

2) **Cài đặt PostgreSQL version >= 10.0**
3) **Cài đặt RabbitMQ**
4) **Cài đặt Elasticsearch version >= 6.0**
5) **Cài đặt Docker và Docker Compose**
6) **Add thư mục 'eureka-server' vào solution: lấy ở file nén** *eureka-server.rar*
7) **Add thư mục 'postgres' vào solution (***lấy ở file nén postgres.rar***)** và tạo các CSDL theo hướng dẫn trong file createdatabases.sql (nằm trong thư mục postgres) trong Hệ quản trị CSDL PostgreSQL.
8) **Add thư mục 'scripts' vào solution: lấy ở file nén** *scripts.rar*
9) **Add thư mục 'Web' vào solution: lấy ở file nén** *Web.rar*
10) **Add thư mục 'BlazorWasmClient' vào solution: lấy ở file nén** *BlazorWasmClient.rar*

11) **Add thư mục 'AgentPortalApiGateway' vào solution: lấy ở file nén** *AgentPortalApiGateway.rar*

CÁC YÊU CẦU CỤ THỂ DÀNH CHO SINH VIÊN:

- Đọc toàn bộ nội dung của các thư mục đã add vào solution để hiểu.
- Buid hệ thống: nếu có phát sinh lỗi thì tìm cách sửa lỗi cho đến khi hệ thống được build thành công.

Nộp sản phẩm:

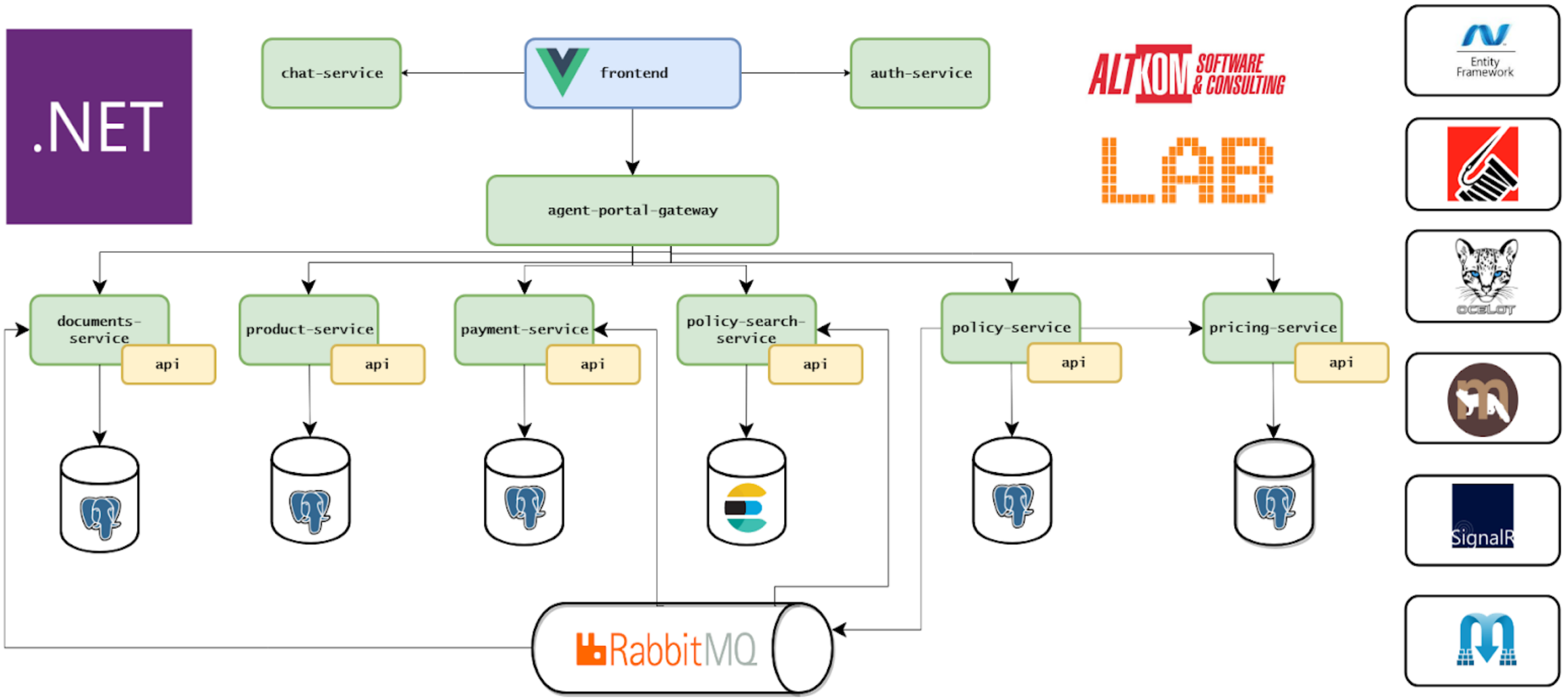
- *Sản phẩm #1:* Thầy sẽ dùng email caotxuan@gmail.com truy cập vào github để kiểm tra code và chấm điểm. (trường hợp này điểm tối đa chỉ đạt 10 điểm). Trường hợp chưa tạo được dự án trên github thì nén project, đưa lên drive và nộp link lên elearning để thầy vào chấm điểm. (trường hợp này điểm tối đa chỉ đạt 8 điểm)
- *Sản phẩm #2: Nộp trên elearniing:* Nộp các file back up các CSDL trong PostgreSQL để thầy có thể restore các file này vào CSDL trên máy tính của thầy để kiêm tra việc có cài đặt được CSDL hay chưa.
- *Sản phẩm #3:* Chụp màn hình kết quả build và nộp lại hình ảnh này. Càng ít lỗi khi build thì càng điểm cao.

Thời hạn nộp: trước 18h00 hôm nay (ngày 11/10/2026).

./.

PHỤ LỤC:

KIẾN TRÚC THAM KHẢO THEO YÊU CẦU CỦA BÀI THỰC HÀNH SỐ 3



Hệ thống bao gồm:

1) **Giao diện người dùng (Frontend)** – một ứng dụng trang đơn (Single Page Application) sử dụng VueJS cung cấp cho các đại lý bảo hiểm khả năng lựa chọn sản phẩm phù hợp cho khách hàng, tính toán giá cả, tạo báo giá và hoàn tất quy trình bán hàng bằng cách chuyển đổi báo giá thành hợp đồng bảo hiểm. Ứng dụng này cũng cung cấp các chức năng tìm kiếm và xem thông tin về hợp đồng và báo giá. Giao diện người dùng giao tiếp với các dịch vụ phụ trợ thông qua cổng API.
2) **Cổng API Đại lý (Agent Portal API Gateway)** – là một microservice đặc biệt với mục đích chính là che giấu sự phức tạp của cấu trúc dịch vụ văn phòng phụ trợ khỏi ứng dụng khách. Thông thường, chúng ta tạo một cổng API riêng cho mỗi ứng dụng khách. Trong trường hợp trong tương lai chúng ta thêm ứng dụng di động Xamarin vào hệ thống, chúng ta sẽ cần xây dựng một cổng API riêng cho nó. Cổng API cũng cung cấp rào cản bảo mật và không cho phép các yêu cầu chưa được xác thực được chuyển đến các dịch vụ phụ trợ. Một ứng dụng phổ biến khác của cổng API là tổng hợp nội dung từ nhiều dịch vụ.

3) **Dịch vụ xác thực (Auth Service)** – một dịch vụ chịu trách nhiệm xác thực người dùng. Hệ thống bảo mật của chúng ta sẽ dựa trên mã thông báo JWT. Sau khi người dùng xác thực chính xác, dịch vụ xác thực sẽ cấp một mã thông báo được sử dụng để kiểm tra quyền của người dùng và các sản phẩm có sẵn.
4) **Dịch vụ Chat (Chat Service)** – một dịch vụ sử dụng SignalR để cho phép các nhân viên hỗ trợ trò chuyện với nhau.
5) **Dịch vụ Định giá (Pricing Service)** – một dịch vụ chịu trách nhiệm tính toán giá cho sản phẩm bảo hiểm cụ thể dựa trên các tham số của nó.
6) **Dịch vụ Chính sách (Policy Service)** – dịch vụ chính trong hệ thống của chúng ta. Nó chịu trách nhiệm tạo ra các gói bảo hiểm và chính sách. Nó sử dụng Dịch vụ Định giá thông qua các cuộc gọi HTTP REST để tính toán giá. Sau khi một chính sách được tạo, nó sẽ gửi sự kiện không đồng bộ đến bus sự kiện (RabbitMQ) để các dịch vụ khác có thể phản hồi.
7) **Dịch vụ Tìm kiếm Chính sách (Policy Search Service)** – mục đích duy nhất của dịch vụ này là cung cấp chức năng tìm kiếm. Dịch vụ này đăng ký các sự kiện liên quan đến vòng đời của chính sách và lập chỉ mục chính sách đã cho trong ElasticSearch để cung cấp khả năng tìm kiếm nâng cao.
8) **Dịch vụ Thanh toán (Payment Service)** – dịch vụ này chịu trách nhiệm quản lý các hoạt động tài chính liên quan đến chính sách. Nó đăng ký các sự kiện liên quan đến chính sách, tạo tài khoản chính sách khi chính sách được tạo, đăng ký các khoản thanh toán dự kiến. Nó cũng có một tác vụ nền rất đơn giản để phân tích tệp chứa các khoản thanh toán đến và gán chúng vào tài khoản chính sách phù hợp.
9) **Dịch vụ Sản phẩm (Product Service)** – đây là danh mục sản phẩm. Nó cung cấp thông tin cơ bản về từng sản phẩm bảo hiểm và các thông số có thể tùy chỉnh khi tạo báo giá cho khách hàng.
10) **Dịch vụ Tài liệu (Document Service)** – dịch vụ này sử dụng JS Report để tạo chứng chỉ PDF.

Mỗi microservice được xây dựng xung quanh ngữ cảnh giới hạn (bounded context) theo thuật ngữ DDD. Bạn cũng có thể thấy rằng chúng hợp tác để đạt được mục tiêu kinh doanh chính – bán sản phẩm bảo hiểm cho khách hàng cuối cùng.